

Module 2 Structured Note: Finding String Length and Changing Case

1. What this module is about

Module 2 teaches three main string operations in C/C++:

1. Finding the length of a string
2. Changing uppercase letters to lowercase
3. Changing lowercase letters to uppercase
4. Toggling a mixed-case string

The main skill in this module is simple but very important:

Start from index `0`, move through the string one character at a time, and stop when you reach `\0`.

In C-style strings, `\0` marks the end of the string. So every string operation in this module depends on checking:

```
S[i] != '\0'
```

2. Required idea from Module 1

Before learning Module 2, remember this from Module 1:

A C-style string is a character array that ends with `\0`.

Example:

```
char S[] = "welcome";
```

Memory view:

```
Index:  0 1 2 3 4 5 6 7
Value:  w e l c o m e \0
```

The visible characters are:

```
w e l c o m e
```

The final `\0` is not part of the visible word. It only tells C where the string ends.

3. Array capacity vs string length

A common beginner mistake is thinking that the size of the array is the same as the length of the string.

Example:

```
char S[20] = "welcome";
```

Here:

```
Array capacity = 20  
String length = 7
```

Why is the string length `7`?

Because the string is:

```
w e l c o m e \0
```

There are 7 visible characters before `\0`.

So:

String length means the number of characters before `\0`, not the total size of the array.

Part A: Finding the length of a string

4. Meaning of string length

The length of a string is the number of actual characters before the null terminator.

Example:

```
char S[] = "welcome";
```

Memory:

```
Index:  0 1 2 3 4 5 6 7
Value:  w e l c o m e \0
```

The length is:

7

Because there are 7 characters before `\0`.

5. Basic logic for finding length

To find the length:

1. Start with index `i = 0`
2. Check whether `S[i]` is not `\0`
3. If it is not `\0`, move to the next index
4. Stop when `S[i] == '\0'`
5. The value of `i` is the length

In simple words:

Keep moving forward until the string ends. The position where you stop is the length.

6. Code for finding string length

```
#include <stdio.h>

int main() {
    char S[] = "welcome";
    int i;

    for (i = 0; S[i] != '\0'; i++) {
        // Empty body: only moving forward
    }

    printf("Length is %d\n", i);
}
```

```
    return 0;
}
```

Output:

Length is 7

7. Why the loop body is empty

The loop is:

```
for (i = 0; S[i] != '\0'; i++) {
    // Empty body
}
```

This loop does not need to do anything inside the body.

Why?

Because the purpose of the loop is only to move `i` forward until it reaches `\0`.

So this loop means:

Start `i` at 0.
As long as `S[i]` is not `\0`, increase `i`.
When `S[i]` becomes `\0`, stop.

At the end, `i` contains the length.

8. Dry run: finding length of "welcome"

String:

welcome

Memory:

Index: 0 1 2 3 4 5 6 7
Value: w e l c o m e \0

Trace:

i	S[i]	Is S[i] != '\0'?	Action
0	w	Yes	Continue
1	e	Yes	Continue
2	l	Yes	Continue
3	c	Yes	Continue
4	o	Yes	Continue
5	m	Yes	Continue
6	e	Yes	Continue
7	\0	No	Stop

The loop stops at index 7.

Therefore:

Length = 7

9. String length as a function

Instead of writing the same length logic again and again, we can create a function.

```
#include <stdio.h>

int length(char S[]) {
    int i;

    for (i = 0; S[i] != '\0'; i++) {
        // Move until null terminator
    }

    return i;
}
```

```
int main() {  
    char name[] = "welcome";  
  
    printf("Length is %d\n", length(name));  
  
    return 0;  
}
```

Output:

```
Length is 7
```

This function manually calculates length without using `strlen()`.

Part B: Uppercase and lowercase in ASCII

10. ASCII values of uppercase letters

Uppercase letters have ASCII values from `65` to `90`.

```
A = 65  
B = 66  
C = 67  
...  
Z = 90
```

So to check whether a character is uppercase, we can write:

```
if (ch >= 65 && ch <= 90)
```

A more readable version is:

```
if (ch >= 'A' && ch <= 'Z')
```

Both mean the same thing.

11. ASCII values of lowercase letters

Lowercase letters have ASCII values from 97 to 122.

```
a = 97  
b = 98  
c = 99  
...  
z = 122
```

So to check whether a character is lowercase, we can write:

```
if (ch >= 97 && ch <= 122)
```

A more readable version is:

```
if (ch >= 'a' && ch <= 'z')
```

12. Difference between uppercase and lowercase letters

Compare uppercase A and lowercase a:

```
'A' = 65  
'a' = 97
```

Difference:

```
97 - 65 = 32
```

So:

```
Uppercase letter + 32 = lowercase letter  
Lowercase letter - 32 = uppercase letter
```

Examples:

```
'A' + 32 = 'a'  
'B' + 32 = 'b'  
'C' + 32 = 'c'  
  
'a' - 32 = 'A'  
'b' - 32 = 'B'  
'c' - 32 = 'C'
```

This `32` difference is the main trick used for case conversion.

Part C: Converting uppercase to lowercase

13. Goal

Suppose we have:

```
char A[] = "WELCOME";
```

We want to convert it into:

```
welcome
```

Each uppercase letter must become lowercase.

Since uppercase to lowercase means adding `32`, we use:

```
A[i] = A[i] + 32;
```

But we should only do this if `A[i]` is uppercase.

14. Uppercase to lowercase code using ASCII numbers

```
#include <stdio.h>  
  
int main() {  
    char A[] = "WELCOME";  
    int i;
```

```

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 65 && A[i] <= 90) {
            A[i] = A[i] + 32;
        }
    }

    printf("%s\n", A);

    return 0;
}

```

Output:

```
welcome
```

15. Uppercase to lowercase code using characters

This version is easier to read:

```

#include <stdio.h>

int main() {
    char A[] = "WELCOME";
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'A' && A[i] <= 'Z') {
            A[i] = A[i] + 32;
        }
    }

    printf("%s\n", A);

    return 0;
}

```

Output:

```
welcome
```

This is usually better for learning because it clearly shows that we are checking letters, not random numbers.

16. Dry run: converting "WELCOME" to lowercase

Initial string:

WELCOME

Memory:

Index: 0 1 2 3 4 5 6
Value: W E L C O M E \0

Trace:

i	Character	Uppercase?	Operation	Result
0	W	Yes	W + 32	w
1	E	Yes	E + 32	e
2	L	Yes	L + 32	l
3	C	Yes	C + 32	c
4	O	Yes	O + 32	o
5	M	Yes	M + 32	m
6	E	Yes	E + 32	e
7	\0	Stop	Stop	Stop

Final string:

welcome

Part D: Converting lowercase to uppercase

17. Goal

Suppose we have:

```
char A[] = "welcome";
```

We want:

```
WELCOME
```

Since lowercase to uppercase means subtracting 32, we use:

```
A[i] = A[i] - 32;
```

But we should only do this if `A[i]` is lowercase.

18. Lowercase to uppercase code

```
#include <stdio.h>

int main() {
    char A[] = "welcome";
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'a' && A[i] <= 'z') {
            A[i] = A[i] - 32;
        }
    }

    printf("%s\n", A);

    return 0;
}
```

Output:

```
WELCOME
```

19. Dry run: converting "welcome" to uppercase

Initial string:

welcome

Memory:

Index: 0 1 2 3 4 5 6
Value: w e l c o m e \0

Trace:

i	Character	Lowercase?	Operation	Result
0	w	Yes	w - 32	W
1	e	Yes	e - 32	E
2	l	Yes	l - 32	L
3	c	Yes	c - 32	C
4	o	Yes	o - 32	O
5	m	Yes	m - 32	M
6	e	Yes	e - 32	E
7	\0	Stop	Stop	Stop

Final string:

WELCOME

Part E: Toggling case

20. Meaning of toggling case

Toggling means changing the case of each letter:

Uppercase -> lowercase
Lowercase -> uppercase
Digits -> unchanged

Symbols -> unchanged
Spaces -> unchanged

Example:

wE1CoMe5

After toggling:

We1cOmE5

21. Toggle case rule

For each character:

1. If it is uppercase, add 32
2. Else if it is lowercase, subtract 32
3. Otherwise, leave it unchanged

In simple words:

Change letters only. Ignore digits and symbols.

22. Toggle case code

```
#include <stdio.h>

int main() {
    char A[] = "wE1CoMe5";
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'A' && A[i] <= 'Z') {
            A[i] = A[i] + 32;
        } else if (A[i] >= 'a' && A[i] <= 'z') {
            A[i] = A[i] - 32;
        }
    }

    printf("%s\n", A);
}
```

```
    return 0;
}
```

Output:

We1c0mE5

23. Dry run: toggling "wE1CoMe5"

Initial string:

wE1CoMe5

Trace:

i	Character	Type	Operation	Result
0	w	Lowercase	w - 32	W
1	E	Uppercase	E + 32	e
2	1	Digit	No change	1
3	C	Uppercase	C + 32	c
4	o	Lowercase	o - 32	O
5	M	Uppercase	M + 32	m
6	e	Lowercase	e - 32	E
7	5	Digit	No change	5
8	\0	End	Stop	Stop

Final string:

We1c0mE5

Part F: Why range checks are important

24. Do not add or subtract 32 blindly

This is wrong:

```
A[i] = A[i] + 32;
```

if you apply it to every character.

Why?

Because not every character is uppercase.

Example:

```
'1' + 32
```

This does not produce a useful letter.

Also:

```
'a' + 32
```

This is wrong because `a` is already lowercase.

So always check the range first.

25. Correct range checks

To check uppercase:

```
if (A[i] >= 'A' && A[i] <= 'Z')
```

To check lowercase:

```
if (A[i] >= 'a' && A[i] <= 'z')
```

These checks protect digits, symbols, and spaces from being changed incorrectly.

26. Why `else if` is used in toggling

In the toggle program, we use:

```
if (A[i] >= 'A' && A[i] <= 'Z') {  
    A[i] = A[i] + 32;  
} else if (A[i] >= 'a' && A[i] <= 'z') {  
    A[i] = A[i] - 32;  
}
```

The `else if` means:

If the character was uppercase and already changed, do not check it again as lowercase.

This keeps the logic clean and avoids unnecessary checking.

Part G: In-place transformation

27. What does in-place mean?

In-place means we change the same array directly instead of creating a new array.

Example:

```
char A[] = "WELCOME";
```

After conversion, the same array becomes:

```
welcome
```

We do not create another array like `B[]`.

28. Use a mutable character array

For in-place changes, use:

```
char A[] = "WELCOME";
```

This creates a modifiable character array.

Avoid this for in-place changes:

```
char *A = "WELCOME";
```

A string literal may not be safely modifiable.

So for beginner code in this module, use:

```
char A[] = "WELCOME";
```

Part H: Complexity analysis

29. Time complexity

All operations in Module 2 scan the string once.

If the string has N characters, the loop visits each character one time.

So the time complexity is:

```
 $O(N)$ 
```

This applies to:

1. Finding length
2. Uppercase to lowercase
3. Lowercase to uppercase
4. Toggling case

30. Space complexity

These operations use only a few variables like:

```
int i;
```

They do not create another array.

So the extra space complexity is:

$O(1)$

This means constant extra space.

31. Complexity table

Operation	Time Complexity	Extra Space
Find string length	$O(N)$	$O(1)$
Convert uppercase to lowercase	$O(N)$	$O(1)$
Convert lowercase to uppercase	$O(N)$	$O(1)$
Toggle mixed-case string	$O(N)$	$O(1)$

Part I: Common mistakes

32. Mistake 1: Thinking array size is string length

Wrong idea:

```
char S[20] = "welcome";
```

Length = 20

Correct idea:

Length = 7

The array has 20 spaces, but the actual string ends at `\0`.

33. Mistake 2: Not checking for `\0`

Bad loop:

```
for (i = 0; i < 20; i++)
```

Better loop:

```
for (i = 0; S[i] != '\0'; i++)
```

Why?

Because the string may end before the full array capacity is used.

34. Mistake 3: Adding 32 to every character

Bad:

```
A[i] = A[i] + 32;
```

Better:

```
if (A[i] >= 'A' && A[i] <= 'Z') {  
    A[i] = A[i] + 32;  
}
```

Only uppercase letters should have 32 added.

35. Mistake 4: Subtracting 32 from every character

Bad:

```
A[i] = A[i] - 32;
```

Better:

```
if (A[i] >= 'a' && A[i] <= 'z') {  
    A[i] = A[i] - 32;  
}
```

Only lowercase letters should have 32 subtracted.

36. Mistake 5: Changing string literals directly

Avoid:

```
char *A = "WELCOME";  
A[0] = 'w';
```

Use:

```
char A[] = "WELCOME";  
A[0] = 'w';
```

The second version creates a modifiable character array.

Part J: Main code patterns to remember

37. Pattern 1: Loop through a string

```
for (i = 0; S[i] != '\0'; i++) {  
    // process S[i]  
}
```

This is the most important pattern in Module 2.

38. Pattern 2: Find length

```
int i;  
  
for (i = 0; S[i] != '\0'; i++) {  
}
```

```
printf("Length is %d", i);
```

39. Pattern 3: Uppercase to lowercase

```
for (i = 0; A[i] != '\0'; i++) {  
    if (A[i] >= 'A' && A[i] <= 'Z') {  
        A[i] = A[i] + 32;  
    }  
}
```

40. Pattern 4: Lowercase to uppercase

```
for (i = 0; A[i] != '\0'; i++) {  
    if (A[i] >= 'a' && A[i] <= 'z') {  
        A[i] = A[i] - 32;  
    }  
}
```

41. Pattern 5: Toggle case

```
for (i = 0; A[i] != '\0'; i++) {  
    if (A[i] >= 'A' && A[i] <= 'Z') {  
        A[i] = A[i] + 32;  
    } else if (A[i] >= 'a' && A[i] <= 'z') {  
        A[i] = A[i] - 32;  
    }  
}
```

Part K: Full combined example

42. Program that finds length and toggles case

```
#include <stdio.h>

int main() {
    char A[] = "wE1CoMe5";
    int i;

    for (i = 0; A[i] != '\0'; i++) {
        // This loop finds the length
    }

    printf("Length is %d\n", i);

    for (i = 0; A[i] != '\0'; i++) {
        if (A[i] >= 'A' && A[i] <= 'Z') {
            A[i] = A[i] + 32;
        } else if (A[i] >= 'a' && A[i] <= 'z') {
            A[i] = A[i] - 32;
        }
    }

    printf("Toggled string is %s\n", A);

    return 0;
}
```

Output:

```
Length is 8
Toggled string is We1cOmE5
```

Part L: Practice questions

43. Practice 1: Find length manually

Find the length of:

```
char S[] = "Code";
```

Memory:

```
C o d e \0
```

Answer:

```
4
```

44. Practice 2: Convert to lowercase

Convert:

```
WELCOME
```

Expected output:

```
welcome
```

Hint:

```
Uppercase + 32 = lowercase
```

45. Practice 3: Convert to uppercase

Convert:

```
welcome
```

Expected output:

```
WELCOME
```

Hint:

Lowercase - 32 = uppercase

46. Practice 4: Toggle case

Toggle:

Data123!

Expected output:

dATA123!

Explanation:

Character	Result
D	d
a	A
t	T
a	A
1	unchanged
2	unchanged
3	unchanged
!	unchanged

47. Practice 5: Explain the mistake

What is wrong with this?

```
A[i] = A[i] + 32;
```

Answer:

This is wrong if we do it for every character because only uppercase letters should have 32 added. Lowercase letters, digits, spaces, and symbols should not be changed this way.

Part M: Final summary

48. Key points to remember

1. A C-style string ends with `\0`.
 2. String length is the number of characters before `\0`.
 3. Array capacity is not the same as string length.
 4. To find length, move through the string until `\0`.
 5. Uppercase letters are from `'A'` to `'Z'`.
 6. Lowercase letters are from `'a'` to `'z'`.
 7. Uppercase to lowercase means adding 32.
 8. Lowercase to uppercase means subtracting 32.
 9. Always use range checks before changing case.
 10. Toggling means uppercase becomes lowercase and lowercase becomes uppercase.
 11. Digits, symbols, and spaces should usually remain unchanged during toggling.
 12. These operations are usually done in-place using a mutable character array.
 13. Each operation takes $O(N)$ time.
 14. Each operation uses $O(1)$ extra space.
-

49. Final mental model

For every operation in Module 2, think like this:

```
Start at index 0.  
Look at the current character.  
If it is not \0, process it.  
Move to the next character.  
Stop when \0 is found.
```

For length:

```
Just count characters before \0.
```

For uppercase to lowercase:

```
If character is uppercase, add 32.
```

For lowercase to uppercase:

```
If character is lowercase, subtract 32.
```

For toggling:

```
Uppercase -> lowercase  
Lowercase -> uppercase  
Other characters -> unchanged
```

50. Module 2 checkpoint

You are ready for Module 3 if you can do these without help:

1. Explain why `"welcome"` has length `7`, not `8`.
2. Write a loop that stops at `\0`.
3. Find string length without using `strlen()`.
4. Explain why uppercase and lowercase letters differ by `32` in ASCII.
5. Convert uppercase letters to lowercase.
6. Convert lowercase letters to uppercase.
7. Toggle a mixed-case string like `wE1CoMe5`.
8. Leave digits and symbols unchanged during toggling.
9. State the time complexity as `O(N)`.
10. State the extra space complexity as `O(1)`.